

Assignment

Optimization Techniques

(UMA035)

by
Ikansh Mahajan (102303754)

submitted to
Mr. Sunny Kumar



THAPAR INSTITUTE
OF ENGINEERING & TECHNOLOGY
(Deemed to be University)

Department of Mathematics
TIET, Patiala

Experiment: Dual Simplex in MATLAB

```
%{  
    MATLAB CODE FOR SOLVING LPP USING DUAL SIMPLEX METHOD  
  
    Min Z = 5x1 + 6x2  
    s.t.  
        x1 + x2 >= 2,  
        4x1 + x2 >= 4,  
        x1, x2 >= 0  
  
    Converting to canonical maximisation form,  
    Max Z = -5x1 - 6x2  
    s.t.  
        -x1 - x2 <= -2,  
        -4x1 - x2 <= -4,  
        x1, x2 >= 0  
  
    Converting to standard form,  
    Max Z = -5x1 - 6x2  
    s.t.  
        -x1 - x2 + s1 = -2,  
        -4x1 - x2 + s2 = -4,  
        x1, x2 >= 0  
  
    Initial BFS is infeasible (RHS negative after conversion),  
    hence Dual Simplex is applied to move from optimal to feasible soln!  
%}  
  
format short  
clear all  
clc  
  
%% Phase 1: Input Parameters  
Variables = {'x_1', 'x_2'};  
C = [-5 -6];  
Info = [-1 -1; -4 -1];  
b = [-2; -4];  
  
[m, n] = size(Info); %% m = No of constraints/slacks; n = No of vars  
%% Phase 2: Add slack vars to variable, table & cost coeffs  
for i = 1:m  
    Variables{end+1} = ['s_' num2str(i)];  
end  
Variables{end+1} = 'Sol';  
  
s = eye(m);  
A = [Info s b];
```

```

Cost = zeros(1, size(A,2));
Cost(1: n) = C;

%% Phase 3: Find initial BV = indices of slack vars!
BV = n+1: size(A, 2)-1;

%% Phase 4: Find Zj - Cj
ZjCj = Cost(BV)*A - Cost;

%% Phase 5: Print current simplex table
ZjC = [ZjCj; A];
SimpTable = array2table(ZjC);
SimpTable.Properties.VariableNames = Variables;
disp(SimpTable);

%% Phase 6: DUAL-SIMPLEX METHOD START
RUN = true;
while RUN
    %% Step-1: Check if soln is feasible
    Sol = A(:, end);
    if any(Sol < 0)
        fprintf('Current soln is NOT FEASIBLE.\n')
        fprintf('\n =====The Next Iteration Results===== \n')
        fprintf('Old Basic Variables (BV) = ')
        disp(Variables(BV))
        %% Step-2: Find the leaving variable
        [LeaVal,pvt_row]=min(Sol);
        fprintf('Most negative element in Sol is %d corresponding to Leaving Row %d.\n',
LeaVal, pvt_row);
        fprintf('=> Leaving Variable = %s\n', Variables{BV(pvt_row)});
        %% Step-3: Find the entering variable using min ratio (Different from Simplex)
        Row = A(pvt_row, 1:end-1);
        Zj = ZjCj(:, 1:end-1);
        if all(Row >= 0)
            error('No negative entries in pivot row, INFEASIBLE SOLUTION.')
        else
            for i=1:size(Row,2)
                if Row(i) < 0
                    ratio(i) = abs(Zj(i)./Row(i));
                else
                    ratio(i) = inf;
                end
            end
            % Calculating minimum Ratio
            [MinRatio, pvt_col]=min(ratio);

```

```

        fprintf('Minimum ratio in Zj/Row is %d corresponding to Entering Column
%d.\n',MinRatio, pvt_col);
        fprintf('=> Entering Variable = %s\n', Variables{pvt_col});
    end
    %% Step-4: Update BV
    BV(pvt_row)=pvt_col;
    fprintf('New Basic Variables (BV) =')
    disp(Variables(BV))
    %% Step-5: Update table for next iteration
    % Pivot key
    pvt_key=A(pvt_row, pvt_col);
    % Update the Table for next Iteration
    A(pvt_row,:) = A(pvt_row,:) ./ pvt_key;
    for i = 1:size(A,1)
        if i ~= pvt_row
            A(i,:) = A(i,:) - A(i,pvt_col) .* A(pvt_row,:);
        end
    end
    ZjCj = Cost(BV)*A - Cost;
    %% Step-6: Print current simplex table
    ZjC = [ZjCj; A];
    SimpTable = array2table(ZjC);
    SimpTable.Properties.VariableNames = Variables;
    disp(SimpTable);
else
    RUN = false;
    fprintf('====*****\n')
    fprintf('Current soln is FEASIBLE & OPTIMAL.\n')
    fprintf('====*****\n')
end
end

%% Phase 7: Print Final Optimal Solution
FINAL_BFS = zeros(1,size(A,2));
FINAL_BFS(BV) = A(:, end); % To store the BFS
FINAL_BFS(end) = sum(FINAL_BFS.*Cost); % Compute value of Z

OptimalBFS = array2table(FINAL_BFS);
OptimalBFS.Properties.VariableNames = Variables;
disp(OptimalBFS);

```

OUTPUT

x_1	x_2	s_1	s_2	Sol
5	6	0	0	0
-1	-1	1	0	-2
-4	-1	0	1	-4

Current soln is NOT FEASIBLE.

=====The Next Iteration Results=====

Old Basic Variables (BV) = {'s_1'} {'s_2'}

Most negative element in Sol is -4 corresponding to Leaving Row 2.

=> Leaving Variable = s_2

Minimum ratio in Zj/Row is 1.250000e+00 corresponding to ENTERing Column 1.

=> Entering Variable = x_1

New Basic Variables (BV) = {'s_1'} {'x_1'}

x_1	x_2	s_1	s_2	Sol
0	4.75	0	1.25	-5
0	-0.75	1	-0.25	-1
1	0.25	0	-0.25	1

Current soln is NOT FEASIBLE.

=====The Next Iteration Results=====

Old Basic Variables (BV) = {'s_1'} {'x_1'}

Most negative element in Sol is -1 corresponding to Leaving Row 1.

=> Leaving Variable = s_1

Minimum ratio in Zj/Row is 5 corresponding to ENTERing Column 4.

=> Entering Variable = s_2

New Basic Variables (BV) = {'s_2'} {'x_1'}

x_1	x_2	s_1	s_2	Sol
0	1	5	0	-10
0	3	-4	1	4
1	1	-1	0	2

=====*****=====

Current soln is FEASIBLE & OPTIMAL.

=====*****=====

x_1	x_2	s_1	s_2	Sol
2	0	0	4	-10

>>